

DRL Router: Distributional Reinforcement Learning-Based Router for Reliable Shortest Path Problems

Hongliang Guo^{*†} 

Is with College of Computer Science,
Sichuan University (SCU),
Chengdu, Sichuan 610065,
China. E-mail: scott0793@gmail.com

Wenda Sheng[†] 

Is with the University of Electronic
Science and Technology of China,
Chengdu 611731, China.
E-mail: ningenpre@gmail.com

Chen Gao[†] 

Is with ETH Zurich, 8092
Zurich, Switzerland.
E-mail: chengao982@gmail.com

Yaochu Jin[†] 

Is with Bielefeld University,
33619 Bielefeld, Germany.
E-mail: yaochu.jin@uni-bielefeld.de

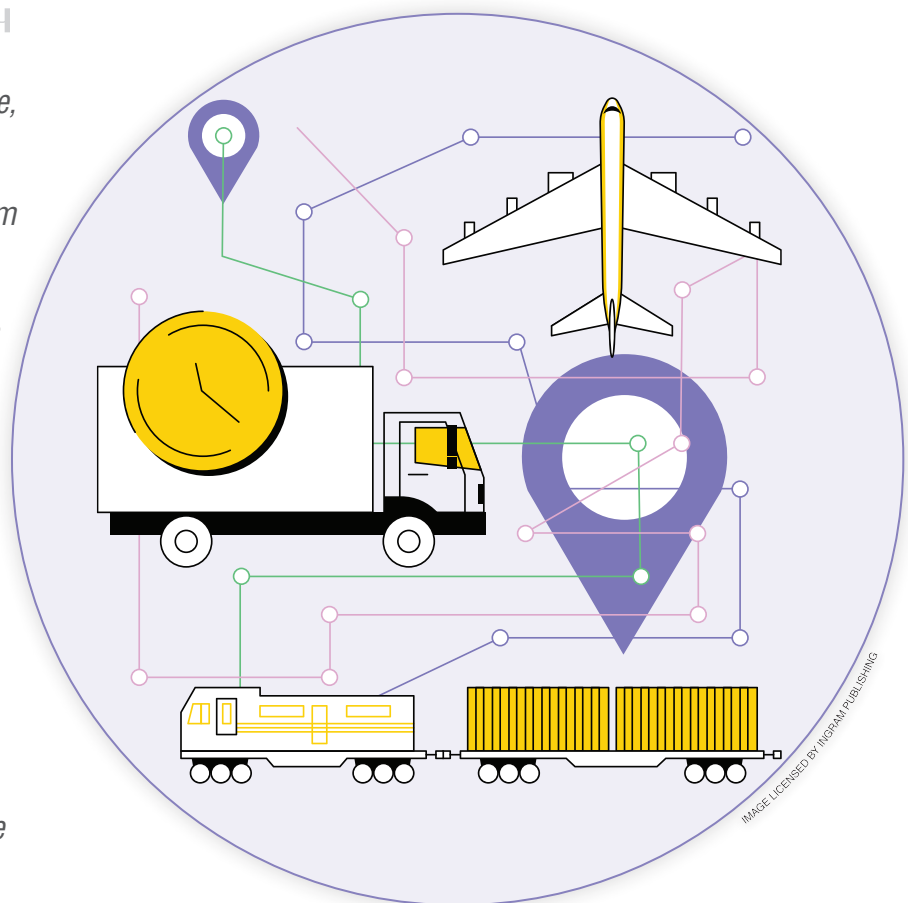


IMAGE LICENSED BY IINGRAM PUBLISHING

Path planning under uncertainty serves as one of the most fundamental services of an intelligent transportation system. While the least expected time (LET) has been deemed the most popular and simplest path planning objective in stochastic transportation networks, in reality, a large portion of risk-averse travelers prefer a reliable shortest path

(RSP) to a LET one [1]. Many definitions of the path's reliability exist in the RSP community; for example, the term *reliability* in the path planning domain can be defined as finding a path or routing policy with the 1) maximal stochastic on-time arrival (SOTA) probability [2]; 2) minimal travel time subject to a high-confidence constraint (α -RSP) [5]; 3) minimal mean and standard deviation combination of travel time

Digital Object Identifier 10.1109/MITS.2023.3265309
Date of current version: 24 April 2023

^{*}Corresponding author. [†]Cofirst authors.

Abstract—This article studies reliable shortest path (RSP) problems in stochastic transportation networks. The term *reliability* in the RSP literature has many definitions, e.g., 1) maximal stochastic on-time arrival probability, 2) minimal travel time with a high-confidence constraint, 3) minimal mean and standard deviation combination, and 4) minimal expected disutility. To the best of our knowledge, almost all state-of-the-art RSP solutions are designed to target one specific RSP objective, and it is very difficult, if not impossible, to adapt them to other RSP objectives. To bridge the gap, this article develops a distributional reinforcement learning (DRL)-based algorithm, namely, DRL-Router, which serves as a universal solution to the four aforementioned RSP problems. DRL-Router employs the DRL method to approximate the full travel time distribution of a given routing policy and then makes improvements with respect to the user-defined RSP objective through a generalized policy iteration scheme. DRL-Router is 1) universal, i.e., it is applicable to a variety of RSP objectives; 2) model free, i.e., it does not rely on well calibrated travel time distribution models; 3) it is adaptive with navigation objective changes; and 4) fast, i.e., it performs real-time decision making. Extensive experimental results and comparisons with baseline algorithms in various transportation networks justify both the accuracy and efficiency of DRL-Router.

(mean-std shortest path) [4]; and 4) minimal expected disutility (MED) [5]. A brief review of the RSP literature, from its objectives and travel time assumptions to its methodologies, is provided in the “Literature Review” section.

Here, we wish to articulate that, to the best of our knowledge, almost all state-of-the-art RSP planning methodologies require at least the first-order and second-order statistic information of the transportation network’s travel time to be available. Meanwhile, it is very difficult, if not impossible, for a specific RSP planning algorithm to apply to all RSP objectives. For example, methods targeting the SOTA problem cannot be easily adapted to solve the α -RSP problem in a straightforward manner. However, in reality, users may dictate their path planning objectives at will and might shift them during runtime. For example, a traveler wants to catch up a flight, and his or her original objective is to arrive at the airport before 5:30 p.m. (the SOTA objective). During navigation, he or she finds that the flight’s departure time is postponed, and in this case, he or she prefers to travel along the cheapest path with high confidence (the α -RSP objective). Therefore, it is desirable to design a “universal” and “adaptive” RSP algorithm that fits a variety of RSP objectives and adapts smoothly to a changed RSP objective.

To this end, this article proposes a distributional reinforcement learning (DRL)-based method for RSP routing (named DRL-Router). When compared with canonical RSP planners, DRL-Router possesses the following characteristics: it is 1) “universal,” i.e., it applies to all the aforementioned RSP planning objectives; 2) “model free,” i.e., it does not rely on travel time distribution models; 3) “adaptive,” i.e., it adapts with user-specific objective changes; and 4) “fast,” i.e., it makes decisions in real time (within 1 s).

Specifically, DRL-Router formulates the RSP planning problem within the Markov decision process (MDP) framework and employs a DRL method to approximate the full travel time distribution of a specific router; it then uses the

generalized policy iteration (GPI) method to update the routing policy and, hence, make improvements toward the optimal one. With the routing policy’s full travel time distribution, DRL-Router is able to calculate the user-designed RSP objective value for each edge and then select the best one to execute. Like canonical reinforcement learning methods, DRL-Router also requires a prohibitive number of interactions with the environment to reach the optimal decision-making policy, which is undesirable. In this article, we invent a LET path-induced warm start method to effectively initialize the full travel time distribution estimations for all state–action pairs and introduce an adaptive double ϵ -greedy exploration strategy to better guide the ego vehicle toward the destination during the initial exploration phase. With the LET path-induced warm start and adaptive double ϵ -greedy exploration strategy, DRL-Router is able to converge to the optimal decision-making strategy efficiently. We explain the details of the two methods in the “Methodology” section.

Here, we wish to point out that through estimating the full travel time distribution, DRL-Router does not necessarily confine its application to the four aforementioned RSP problems. In fact, as long as the navigation objective can be expressed as a function of the full travel time distribution, e.g., a conditional value at risk objective, DRL-Router can be employed to solve the problem. This is the reason that we characterized DRL-Router as a “universal” RSP solution. The contributions of the article are twofold. First, from the transportation application’s perspective, 1) we propose a universal RSP path planner, i.e., DRL-Router, which applies to four completely different RSP objectives; 2) DRL-Router adapts automatically with user-specific RSP objective changes; 3) once training is finished, DRL-Router’s decision-making time is within milliseconds for large-scale transportation networks; and 4) DRL-Router is model free and thus does not need well calibrated travel time distribution parameters as inputs. Second, from the technology innovation perspective, 1) we introduce a LET path-induced warm start method to effectively

initialize DRL-Router, which greatly accelerates the training speed, and 2) we propose an adaptive double- ϵ exploration method to better guide the ego vehicle toward the destination during the initial exploration phase. In this way, the learning process of DRL-Router is much stabler than the default one.

The structure of the article's remaining content is as follows. We present a short literature review of RSP and DRL in the "Literature Review" section, followed by background knowledge, including path planning basics, RSP problem formulations, and DRL, in the "Problem Formulation and DRL Background" section. The "Methodology" section presents DRL-Router, with training time complexity as well as runtime complexity analysis. Experimental results, comparisons, and analysis are provided in the "Simulation Results and Comparisons" section. We finish the article with concluding remarks and improvement directions in the "Conclusion and Future Work" section.

Literature Review

Since this article investigates the RSP problem from the DRL perspective, in this section, we first deliver a brief literature review of the RSP, from its objectives, assumptions, and methodologies to its output formats, and specifically lay down the differences between DRL-Router and canonical learning-based RSP planners. Then, we present the recent research trends for DRL toward the end of the section.

A Brief Review of RSP

RSP has been a hot research topic in the past few decades, Table 1 provides a bird's-eye view of the RSP literature

It is desirable to design a "universal" and "adaptive" RSP algorithm that fits a variety of RSP objectives and adapts smoothly to a changed RSP objective.

works along the taxonomies of 1) reliable navigation objectives, 2) travel time distribution models, 3) solution methodologies, and 4) output formats. Next, we describe the related content in detail along the four aforementioned directions.

RSP Planning Objectives

As we mentioned in the first section, there are many definitions of reliability against travel time variation in the path planning community. We categorize the objectives of an RSP problem into the following four groups, namely, 1) SOTA, 2) α -RSP, 3) minimal mean-std, and 4) MED.

The SOTA RSP, first proposed in [2], finds a path with the maximal probability of on-time arrival at the destination, with a prespecified deadline. Researchers have proposed various algorithms to target the SOTA problem; see [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], and [18] for examples. Recently, there has been an emerging trend of incorporating the original on-time arrival objective with other factors. For example, Manseur et al. propose a novel robust SOTA problem, which takes SOTA as the main objective and considers the alternative path's SOTA performance when random edge breakdowns happen during navigation [6].

The α -RSP, proposed by Chen and Ji, in [3], is to find a path with the minimal high-percentile travel time; i.e., the

Table 1. A summary of the RSP literature.

Objectives	Travel Time Distribution Assumptions		Methodologies	Output Formats
SOTA [2], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18]	With/without a distribution model	Data driven [9], [12], [19], [20], [21], [22]	Mathematical programming [9], [12], [16], [17], [23], [24]	Path-based solution [10], [24], [25], [26]
α -RSP [28], [30], [31], [32], [33], [34]		Full distribution model [8], [16], [24], [27], [28], [29]		
		Distribution model statistics [25], [26], [35]	Dynamic programming [6], [7], [15], [36], [37], [38], [39]	Policy-based solution [6], [7], [16], [22], [36], [41]
Minimal mean-std [21], [22], [24], [25], [26], [35], [39], [41], [42], [43], [44]	Spatial distribution	Spatially independent [7], [22], [35], [40]		
	Spatially correlated [14], [16], [24], [25], [26], [30], [41]	Genetic programming [3], [29], [45], [46]		
MED [5], [47]	Temporal distribution		Stationary [27], [28], [34], [41]	
		Time varying [7], [9], [33], [47], [48]		

Most research works in RSP can be deemed model-based solutions in which the description of the underlying transportation network's travel time distribution is needed a priori.

path's realized travel time is shorter than T with a probability higher than α ; see [18], [30], [31], [32], [33], and [34] for examples. It is worth noting that the LET path can be treated as a special case of the α -RSP, with $\alpha = 0.5$. This means that the traveler wants to minimize the path's 50th percentile travel time, which is the expected travel time, by definition.

The mean-std shortest path problem is to find a routing policy with the minimal combination of the mean and standard deviation [4]. Since the mean-std RSP directly trades off the mean travel time with its variance, it is fitted to the risk-averse decision-making domain, and many researchers have contributed to this research scope [21], [22], [24], [25], [26], [35], [39], [41], [42]. Variations of the mean-std RSP problem include finding a path with 1) the minimal combination of mean and variance, studied in [43]; 2) the minimal mean travel time subject to variance constraints [44]; and 3) only the minimal variance [49], [50].

The MED RSP problem comes from utility theory [51], which copes with humans' decision making against variations, in general (sometimes, the RSP objective is to find a path with the maximal expected utility). We denote an ego path x 's travel time as c_x (a random variable) and define a disutility function u . Then, the path with the minimal expected value of the disutility function, i.e., $\mathbb{E}(u(c_x))$, is defined as the RSP. Depending on the forms of the disutility function, different MED RSP problem can be formulated, e.g., the piecewise linear function [5] and exponential function [47]. It is worth mentioning that the LET path is a special case of the MED RSP problem, where the disutility function is the path's travel time itself; i.e., $u(c_x) = c_x$.

Travel Time Distribution Assumptions

Whether and how to describe the stochastic travel time distributions of the underlying transportation network plays a vital role in designing corresponding algorithms for RSP planning. Next, we describe the related studies in this taxonomy in terms of 1) model-free/model-based methods, 2) the travel time's spatial correlation characteristic, and 3) the travel time's temporal distribution characteristic.

Most research works in RSP can be deemed model-based solutions in which the description of the underlying transportation network's travel time distribution is needed

a priori. Within this group, a few researchers presume that the full distribution model is available and thus propose corresponding methodologies. For example, the (multi-variate) Gaussian distribution [10], [24], [27], [28], [37], [52], (shifted) log-normal distribution [29], [53], Lévy distribution [8], Burr distribution [54], and Weibull distribution

[55] are the most commonly used travel time distribution models (interested audiences are referred to [56] for detailed performance comparisons among different distribution models). Other researchers require only the first-order (mean) and second-order (covariance) statistical information to be available instead of the full distribution model; see [25], [26], and [35] for examples. On the other hand, several research works in RSP are essentially model free in that they do not assume specific distribution models and model statistics to be available. The related research works typically formulate the RSP problem as a data-driven optimization problem and employ off-the-shelf optimization solvers for an exact or approximate solution [9], [12], [19], [20], [21].

For the spatial relationship of the travel time distribution, some researchers presume independent travel time distributions [7], [22], [35], [40], while others target spatially correlated distributions [14], [16], [24], [25], [26], [30], [41]. For the travel time's temporal characteristics, most researchers target stationary travel time distributions, where the travel time distribution does not change with time [27], [28], [41], while there are some research works specifically dealing with time-varying transportation networks [7], [9], [47], [48], [57].

RSP Solution Methodologies

The methodologies for the RSP problem are roughly divided into three branches, namely, 1) mathematical programming (MP)-based methods, 2) dynamic programming (DP)-based methods, and 3) genetic programming (GP)-based methods.

MP-based methods usually formulate the RSP problem into a mathematical optimization problem, and commercial solvers are employed for the optimal solution [9], [12], [23]. For example, Yang formulates the SOTA RSP problem as a mixed-integer linear programming problem and invokes CPLEX (a commercial solver) to solve the problem [9]. The mathematical representation of the RSP problem is succinct and easy to understand, and off-the-shelf solvers are available. However, the efficiency of the solver is usually not good, as the formulated problem almost always falls into the nonlinear integer programming category. Thus, it is very difficult to directly apply MP-based methods to large-scale transportation networks. Aiming at improving the efficiency of MP-based methods, various approximation algorithms are proposed [11], [19], [25], [26], [30]. The approximation algorithms usually

make use of the superefficiency of computing a LET path and use the Lagrangian relaxation method to decompose the originally complex mathematical problem into several easy-to-solve LET path problems.

DP-based methods try to approximate the solution to the RSP problem from the DP perspective and apply label correction and/or path pruning techniques to gradually update each node's value based on the last-step estimated neighbor node values [6], [7], [8], [14], [15], [36], [37], [38], [52]. For example, Prakash proposes a decreasing order of time (DOT) approach to recursively update a node's on-time arrival probability. Based on DOT, a pruning technique is proposed to calculate the a priori SOTA path.

GP-based methods use GP, or evolutionary algorithms, to solve the RSP problem [3], [29], [45], [46]. Since GP is derivative free, the computational complexity depends only on the number of seeds per generation and the number of offspring layers. With this kind of method, one can easily start with a feasible solution and gradually improve toward the optimal one through evolutionary algorithms. Chen et al. propose a moment matching-based hybrid genetic algorithm to search for the α -RSP in stochastic transportation networks with link correlations [28]. The proposed algorithm is validated with real traffic data in Beijing, China.

Besides the three aforementioned main branches of RSP solution methodologies, there is a recent emerging trend of applying learning-based methods to solve a specific RSP problem. Cao et al. formulate the SOTA problem within the MDP framework and propose a practical Q-learning (PQL) method, which uses a multilayer perceptron to approximate reinforcement learning agent's Q values, i.e., the corresponding nodes' on-time arrival probabilities [18]. Guo et al. propose a cascaded temporal difference (CTD) method, which simultaneously estimates the mean and standard deviation of a given routing policy [22] and thereby solves the mean-std shortest path problem. Zhou et al. apply a learning-based method to solve the path following of an under-actuated unmanned surface vehicle with state constraints [58]. The authors employ a critic neural network to estimate the cost function, and the dynamic controller improves its reliability performance by considering the event-triggered condition. DRL-Router also belongs to the learning-based method for RSP solutions. However, it employs DRL to estimate the full distribution of a given routing policy and hence applies to a variety of RSP objectives, including but not limited to SOTA, α -RSP, and the mean-std shortest path.

RSP Planning Output Formats

In general, there are two kinds of solution formats to the RSP problem, namely, path-based solutions [10], [24], [26]

DRL has been shown to be superior to canonical reinforcement learning methods in scenarios where the return's entire distribution plays an important role in decision making.

and policy-based solutions [6], [7], [16], [22], [36], [41], [59]. Path-based solutions yield a nonadaptive path to the ego vehicle, while policy-based solutions reach an adaptive routing policy, which points out merely the next node at each junction point and defers the path selection process thereafter to the time point when the ego vehicle actually reaches the next node. Normally, policy-based solutions need the RSP algorithm to make decisions in real time, i.e., in less than 1 s, in that the ego vehicle cannot bear to wait too long during navigation.

DRL

In canonical reinforcement learning, the agent interacts with the environment, estimates the expected return of a given policy, and makes improvements toward the optimal policy which, by definition, maximizes the expected return [60], while in DRL, the agent estimates the entire distribution of the return instead of its expectation [61]. With that, the DRL agent is able to make more informed decisions. DRL has been shown to be superior to canonical reinforcement learning methods in scenarios where the return's entire distribution plays an important role in decision making.

The pioneering work of DRL was proposed in [61], in which the authors used sampling-based methods to estimate the full distribution of the return. The distributional Bellman equation was established, and a fixed number of intervals (51 in the paper) were used to approximate the return's distribution in a discrete manner. In the following years, researchers made improvements over the original DRL methods by 1) using an adjustable length of intervals to approximate the return's distribution [62], 2) estimating the return's distribution through implicit quantile networks [63], 3) fully parameterizing the return distribution through quantile functions [64], and 4) incorporating an actor-critic network for DRL application in the continuous action application domain [65].

Problem Formulation and DRL Background

In this section, we first introduce the notations used throughout the article and then formulate the four RSP planning problems into mathematical optimization forms. Since we use DRL as a universal solution for the listed RSP problems, we introduce the basics of DRL toward the end of this section.

Notations

In this article, we represent the transportation network as a connected and directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ ($|\mathcal{V}| = n$) refers to the set of vertices and $\mathcal{E}$ ($|\mathcal{E}| = m$) refers to the set of edges. Assume that the travel time c_{ij} of edge $ij \in \mathcal{E}$ is a random variable with mean $\mu_{ij} \geq 0$ and standard deviation σ_{ij} . Let (o, d) be a user-specified origin-destination (OD) pair, and a path is represented by an incidence vector $\mathbf{x} \in \{0, 1\}^m$. We denote the complete set of feasible paths connecting o to d as $\mathcal{P}_{od}$. A list of major notations used in this article is in Table 2.

Problem Formulation

This article aims at proposing a universal solution for all four of the aforementioned RSP planning problems; thus, we present the respective problem formulations of 1) SOTA, 2) α -RSP, 3) the mean-std shortest path, and 4) MED in the following.

Table 2. The primary notations used in this article.

Notation	Description
$\mathcal{G}(\mathcal{V}, \mathcal{E})$	Transportation network with vertices $\mathcal{V}$ and edges $\mathcal{E}$
$\mathcal{V}$	Set of vertices, $ \mathcal{V} = n$
$\mathcal{E}$	Set of edges, $ \mathcal{E} = m$
i, j, o, d	Vertex index, with o representing the origin and d representing the destination, $i, j, o, d \in \mathcal{V}$
ij, kl	Edge index, $ij, kl \in \mathcal{E}$
c_{ij}	Travel time of edge ij (a random variable)
$\mu_{ij}/\sigma_{ij}/\tau_{ij}$	Mean/standard/variance of edge ij 's travel time
$\mathbf{c}, \boldsymbol{\mu}, \boldsymbol{\sigma}, \boldsymbol{\tau}$	Vector whose elements are $c_{ij}, \mu_{ij}, \sigma_{ij},$ and τ_{ij}
$\mathbf{x} \in \{0, 1\}^m$	Incidence vector denoting a path
c_x	Total travel time of path $\mathbf{x}$ (a random variable)
$\mu_x/\sigma_x/\tau_x$	Mean/standard/variance of path $\mathbf{x}$'s travel time
$\mathcal{P}_{od}$	Set of all feasible paths connecting o to d
T	Path planning budget
α	Confidence level
ζ	Reliability coefficient
$u(c)$	Disutility function over the path's travel time
$\mathcal{X}, \mathcal{A}$	State space and action space in MDP
$\mathcal{P}$	State transition function in MDP
s_k, a_k, r_k	State, action, reward at time k ; $s_k \in \mathcal{X}, a_k \in \mathcal{A}$
$Q^\pi(s, a)$	Expected return of policy π starting at (s, a)
$Z^\pi(s, a)$	Full distribution of policy π 's return starting at (s, a)
α_0	Learning rate at time step t
N	Number of basis for Z^π approximation
K	Number of travel time samples per edge

The SOTA Problem

The objective is to find a routing policy that maximizes the probability of reaching the destination node within the given budget T . The mathematical problem can be formulated as

$$\begin{aligned} & \underset{\mathbf{x}}{\text{maximize}} && \mathbb{P}(c_x \leq T) \\ & \text{subject to} && \mathbf{x} \in \mathcal{P}_{od} \end{aligned} \quad (1)$$

where T is the user-specified path planning budget, c_x is the total travel time of path $\mathbf{x}$, and $\mathcal{P}_{od}$ refers to the total set of paths connecting o with d .

The α -RSP Problem

The objective is to find a path that minimizes the travel time at a given confidence level α . The confidence level indicates that the path has α probability of arriving on time. The problem is formulated as

$$\begin{aligned} & \underset{\mathbf{x}, T}{\text{minimize}} && T \\ & \text{subject to} && \mathbb{P}(c_x \leq T) \geq \alpha, \\ & && \mathbf{x} \in \mathcal{P}_{od} \end{aligned} \quad (2)$$

where α is the user-specified confidence level.

The Minimal Mean-Std Problem

The objective is to find a path with the minimal mean-std combination with respect to the travel time. The problem is formulated as

$$\begin{aligned} & \underset{\mathbf{x}}{\text{minimize}} && \mu_x + \zeta \sigma_x \\ & \text{subject to} && \mathbf{x} \in \mathcal{P}_{od} \end{aligned} \quad (3)$$

where ζ is the reliability coefficient and μ_x/σ_x is the mean/std of path $\mathbf{x}$'s travel time.

The MED Problem

In this problem, the traveler specifies a disutility function over the path's instantiated travel time, i.e., $u(c_x)$, and the objective is to find a path with the minimal expected value of the disutility function. The problem is formulated as

$$\begin{aligned} & \underset{\mathbf{x}}{\text{minimize}} && \mathbb{E}(u(c_x)) \\ & \text{subject to} && \mathbf{x} \in \mathcal{P}_{od} \end{aligned} \quad (4)$$

where $\mathbb{E}(X)$ refers to the expectation of random variable X . Many forms of the disutility function can be defined, and the most commonly used ones are the piecewise linear function [5] and exponential disutility function [47].

DRL

DRL, originally proposed in [61], is a special form of reinforcement learning. In DRL, the underlying environment is also modeled as an MDP $(\mathcal{X}, \mathcal{A}, R, \mathcal{P}, \gamma)$, with $\mathcal{X}$ and $\mathcal{A}$ representing the state space and action space, respectively. Here, R is the reward function; $\mathcal{P}$ is the state action

to next-state transition kernel $\mathcal{P}(\cdot|s, a)$, and $\gamma \in [0, 1]$ is the discount factor. A DRL agent interacts with the environment in the standard fashion: at each step k , the agent selects an action $a_k \in \mathcal{A}$ based on its current state $s_k \in \mathcal{X}$; to which the environment responds with a reward r_{k+1} and the next state $s_{k+1} \in \mathcal{X}$.

Different from the canonical reinforcement learning algorithms, which evaluate the expected value of the return under a given policy π , i.e., $V^\pi(s), Q^\pi(s, a)$, DRL proposes a distributional Bellman equation and targets estimating the full distribution of a given policy's return, i.e., $Z^\pi(s), Z^\pi(s, a)$. The distributional Bellman equation is represented as follows:

$$Z^\pi(s_k, a_k) = r_{k+1} + \gamma Z^\pi(s_{k+1}, a_{k+1}). \quad (5)$$

The equal sign in (5) requires that the two distributions are “equivalent” to each other. In DRL, the Wasserstein distance [66] is usually used to describe the distance between the two distributions. In implementation, $Z^\pi(s, a)$ is, in most cases, approximated by a multinomial distribution with a prefixed set of intervals [61], [63]. The objective is to find the best parameter configurations to minimize the Wasserstein distance between the two distributions, and the authors of [61] prove that (5) is a contractive operation, and hence, the related algorithms implementing (5) converge to the optimality.

Methodology

In this section, we present DRL-Router, which serves as a universal learning-based solution for the RSP problem. We first reformulate the RSP planning problem from the MDP framework perspective, then present the two interactive modules within DRL-Router, namely, 1) policy evaluation and 2) policy improvement. DRL-Router's algorithm flow process, computational complexity, and runtime complexity are presented and analyzed toward the end of this section.

RSP Problem Reformulation Within the MDP Framework

DRL-Router formulates the general RSP problem as a sequential decision-making problem within the MDP framework, instead of treating it as monolithic nonconvex mathematical optimization problems as presented in (1)–(4). The key to formulate the RSP planning problem within the MDP framework is to properly map the MDP-related elements, e.g., $\mathcal{X}, \mathcal{A}, R, \mathcal{P}$, and γ , into the path planning-related variables and express the RSP objective with MDP terminologies.

The mapping process is as follows: 1) the state of the vehicle ($s_t \in \mathcal{X}$) is the vehicle's located vertex, e.g., vertex i , and the absorbing state is vertex d ; 2) the action of the vehicle ($a_t \in \mathcal{A}$) is the edge that the vehicle takes, e.g., edge ij ; 3) the immediate reward r_{t+1} is the sampled travel time of edge ij , i.e., c_{ij} ; 4) the state transition func-

tion $\mathcal{P}$ is a deterministic function, which transits the vehicle to the end node of the current executing link, e.g., $s_{t+1} = j$, where $j \in \mathcal{V}$; and 5) the discount factor $\gamma = 1$ in RSP planning. A routing policy π determines, possibly probabilistically, the next-executing edge (a_t) of the vehicle given its current state (s_t), e.g., $a_t = \pi(s_t)$. (The absorbing state in reinforcement learning refers to the state that ends the episode, and it corresponds to the destination (vertex d) in the path planning domain.) The return of policy π starting at (s, a) , i.e., $Z^\pi(s, a)$, is the total travel time distribution of policy π , starting at (s, a) . Similarly, $Z^\pi(s)$ is the total travel time distribution of policy π , starting at state s .

Depending on specific RSP objectives, e.g., SOTA, α -RSP, mean-std, and MED, we can transform the full travel time distribution $Z^\pi(s)$ to the corresponding scalar values (denoted as obj) representing the policy's performance metric: 1) for SOTA, $\text{obj} = \mathbb{P}(Z^\pi(s) \leq T)$; 2) for α -RSP, $\text{obj} = T$, where $\mathbb{P}(Z^\pi(s) \leq T) \geq \alpha$; 3) for mean-std, $\text{obj} = \mathbb{E}(Z^\pi(s)) + \zeta\sigma(Z^\pi(s))$; and 4) for MED, $\text{obj} = \mathbb{E}(u(Z^\pi(s)))$. So far, we have formulated the RSP planning problem within the MDP framework; the objective is to find the best routing policy π^* , which maximizes/minimizes the corresponding performance metric value obj. (For SOTA, the objective is to maximize obj; for all the other RSP objectives, the objective is to minimize obj.) In the following two sections, we present the two modules in DRL-Router, namely, 1) policy evaluation, which estimates $Z^\pi(s)$ for a given policy π , and 2) policy improvement, which updates policy π toward π^* .

Policy Evaluation for DRL-Router

The policy evaluation model in DRL-Router is to estimate the full travel time distribution for all the state–action pairs for a given policy π . The straightforward approach is to run the policy for all the state–action pairs multiple times and use the sampled return (the total travel time) values to approximate the distribution. However, the approach is inefficient in terms of both sampling complexity and computational complexity.

We employ the TD learning [60] scheme to estimate the return distribution of one state–action pair from the bootstrapped distribution estimates of another state–action pair. Note that in reinforcement learning, there are altogether three schemes to estimate the return's distribution, namely, DP, Monte Carlo, and TD. First, DP requires the full travel time distribution model to be available, which restricts its application to model-free use cases. On the other hand, although Monte Carlo is indeed a model-free solution, it needs to wait until the end of an episode for the estimation's update, which is quite inefficient. In practice, we need an algorithm that is able to update the distribution estimates based on bootstrapped estimates from the past. This is exactly what the TD method does for the full

travel time distribution update. Therefore, we select the TD method to estimate the return distribution.

When given a specific transition sample, (s, a, r, s', a') , the update target for $Z^\pi(s, a)$ is $r + Z^\pi(s', a')$. Here, we wish to note that throughout this article, when we add a scalar to a vector, we mean that the scalar is added element-wise to the vector, and the result is also a vector. We have

$$Z_{t+1}^\pi(s, a) \longleftarrow r + Z_t^\pi(s', a'). \quad (6)$$

Since $Z^\pi(s, a)$ is a distribution rather than a scalar, it is not straightforward to apply the TD update method directly. In this article, we adopt and revise the C51 method, originally proposed in [61], to perform updates for Z . Specifically, we divide the full range of Z values for all the state-action pairs into a fixed number (N) of intervals (named *basis*) and use the categorical distribution to approximate Z . Then, the objective is to minimize the Wasserstein distance between $Z_{t+1}^\pi(s, a)$ and $r + Z_t^\pi(s', a')$. We compute the gradient with respect to the Wasserstein distance [66], and $Z_{t+1}^\pi(s, a)$ can be updated as

$$Z_{t+1}^\pi(s, a) = Z_t^\pi(s, a) + \alpha_0(\mathbb{T}(r + Z_t^\pi(s', a')) - Z_t^\pi(s, a)) \quad (7)$$

where $0 \leq \alpha_0 \leq 1$ is the learning rate and $\mathbb{T}(r + Z_t^\pi(s', a'))$ is a function that transforms the discrete distribution representation of $(r + Z_t^\pi(s', a'))$ to fit the distribution basis of $Z^\pi(s, a)$ and hence makes the addition of two distributions possible. Note that, in essence, (7) is to calculate the summation of two vectors, and the plus symbol in (7) is an element-wise summation between the two vectors.

So far, we have presented a TD-like policy evaluation method for the full travel time distribution $[Z^\pi(s, a)]$ estimation. For its application in the path planning domain, we make improvements along two aspects: 1) where to start the distribution basis of $Z^\pi(s, a)$ and 2) involving more than one travel time sample for the update process in (7).

Since the reward $[r(s, a)]$ is an instantiated travel time sample of the traversal link, it must have a lower bound. In this case, $Z^\pi(s, a)$ also has the absolutely smallest value [denoted as $z_{\min}^\pi(s, a)$], which is the summation of all the traversal link's minimal travel time, under policy π until the vehicle reaches the destination vertex d . In this case, one can start the distribution basis of $Z^\pi(s, a)$ at $z_{\min}^\pi(s, a)$, which is reasonable because there is zero probability that, under policy π , the vehicle arrives at the destination with a shorter travel time than $z_{\min}^\pi(s, a)$.

Another characteristic in path planning is that, for each link, one can collect multiple travel time samples. For example, with K travel time samples of a given edge, one can improve the computational process of (7) to the following form:

$$Z_{t+1}^\pi(s, a) = (1 - \alpha_0)Z_t^\pi(s, a) + \frac{\alpha_0}{K} \sum_{k=1}^K \mathcal{T}(r_k + Z_t^\pi(s', a')). \quad (8)$$

Through (8), the policy evaluation process converges faster because more travel time samples are involved in the travel time distribution update process.

Policy Improvement for DRL-Router

In the preceding section, we introduced the policy evaluation module for DRL-Router. The next question is how to improve the policy toward the optimal one for the RSP planning problem. We follow the GPI scheme proposed in [60] to improve policy π toward π^* . As we stated in the "RSP Problem Reformulation Within the MDP Framework" section, with $Z^\pi(s, a)$, one is able to calculate a performance metric value (obj) for a specific RSP problem, e.g., SOTA and α_0 -RSP. Depending on the specific RSP problem, we can update the policy toward the one that maximizes/minimizes the value of obj.

In the following, we first describe how to calculate obj for any given state-action pair for the four RSP problems and then present the policy update process. For the SOTA problem, we have

$$\text{obj}(s, a) = \mathbb{P}(Z^\pi(s, a) \leq T). \quad (9)$$

For the α -RSP problem, we have

$$\text{obj}(s, a) = T \quad (10)$$

where

$$\mathbb{P}(Z^\pi(s, a) \leq T) \geq \alpha.$$

For the mean-std problem, we have

$$\text{obj}(s, a) = \mathbb{E}(Z^\pi(s, a)) + \zeta \sigma(Z^\pi(s, a)) \quad (11)$$

and for the MED problem, we have

$$\text{obj}(s, a) = \mathbb{E}(u(Z^\pi(s, a))). \quad (12)$$

With obj values for all the state-action pairs, we can update policy π to π' in the following way: 1) for the SOTA problem,

$$\pi'(s) = \arg \max_a \text{obj}(s, a) \quad (13)$$

and 2) for the α -RSP problem, minimal mean-std problem, and MED problem,

$$\pi'(s) = \arg \min_a \text{obj}(s, a). \quad (14)$$

This is a greedy approach to update π ; however, in reinforcement learning, due to various sources of uncertainties, one needs to balance exploitation (select the best action based on current estimates) and exploration (explore other action options). In DRL-Router, since the main focus is to propose DRL for the RSP problem, for now, we

do not delve deep into the exploration strategies inside the reinforcement learning process and use only the simplest exploration strategy, i.e., ϵ -greedy, to foster exploration. At each time step, the ego vehicle will select the best action, i.e., $a = \pi'(s)$, with probability $1 - \epsilon$ and select any other action with equal probabilities. Note that we introduce an improved exploration strategy for DRL-Router, namely, adaptive double ϵ -greedy, in the “Adaptive Double ϵ -Greedy Exploration” section.

LET Path-Induced Warm Start of DRL-Router

We have introduced the policy evaluation module and policy improvement module for DRL-Router in the previous two sections, respectively. Theoretically, DRL-Router works with random initialization of the policy and its corresponding travel time distribution Z . However, with random initialization, the ego vehicle needs quite a number of episodes for exploration in DRL-Router’s initial phase. Thus, it is better to endow DRL-Router with a good starting point for both the policy π and its corresponding travel time distribution Z^π .

With modern sensing and communicating technologies, it is easy to construct a database storing real travel time samples of all the edges in the transportation network. Then, the sample mean of the travel time can be easily obtained. In this way, we initialize DRL-Router’s policy π as the LET path from the given vertex to the destination vertex (d), which means that at any vertex s , policy π will select the edge that is contained in the LET path from s to d . With the initialized policy π , we can assign $Z^\pi(s, a)$ as the corresponding LET path’s travel time distribution, which is approximated with the database of travel time samples.

The reason we select a LET path to initialize the travel time distribution estimates is that, in reality, a LET path, in most cases, behaves quite well for any RSP problem. Therefore, using the LET-path induced distribution as the initial travel time distribution estimation is a very good starting point. Moreover, calculating a LET path as well as the corresponding travel time distribution is very easy with the A* and Dijkstra algorithms. In this case, we use the LET-path induced policy to initialize the travel time distribution estimation and then employ the GPI scheme to gradually update the policy toward the optimal one.

Adaptive Double ϵ -Greedy Exploration

In the “Policy Improvement for DRL-Router” section, we introduced the policy improvement procedure for DRL-Router. The canonical ϵ -greedy exploration strategy is employed for exploration. However, we observe that in the initial training stages, with the canonical ϵ -greedy exploration strategy, the ego vehicle tends to fall into a loop, e.g., traveling back and forth between intermediate nodes, which is meaningless. In view of that, we invent an adaptive double ϵ -greedy strategy, which has ϵ_1 probability of

selecting the LET routing policy-referred action, ϵ_2 probability of selecting the reliability metric-guided action, and $1 - \epsilon_1 - \epsilon_2$ probability of selecting the action at uniform randomness. We set $\epsilon_1 = 0.9$ and $\epsilon_2 = 0.05$ and gradually decrease ϵ_1 and increase ϵ_2 as the training episodes increase, e.g., $\epsilon_1 = \epsilon_1 \times 0.99^{\text{Epi}}$, where Epi refers to the number of training episodes. Similar but twisted mechanisms can be applied to update ϵ_2 . We verify the effectiveness of the adaptive double ϵ -greedy exploration strategy in the “Simulation Results and Comparisons” section.

Algorithm Flow Process

With the policy initialization, policy evaluation, and policy improvement modules of DRL-Router, we are ready to deliver DRL-Router’s algorithm flow process, which is displayed in Algorithm 1. Note that DRL-Router is a universal solution for the RSP problem, but the user still needs to specify the type of RSP problem, e.g., SOTA and α -RSP, and corresponding parameters, e.g., path planning budget T and user confidence level α . Depending on the type of RSP problem, one of the equations from (9) to (12) will be invoked for the performance metric (obj) computation, and DRL-Router will perform the policy update thereafter. DRL-Router is essentially looping from the policy evaluation module (line 4 to line 6) to the policy iteration module (line 7 to line 10).

In Algorithm 1, we let the ego vehicle “stick” with the current policy for the entire episode and perform the policy evaluation and policy improvement thereafter. This is because we

Algorithm 1. DRL-Router’s Algorithm Flow Process.

Input: 1) destination d ; 2) RSP problem-specific parameters $T/\alpha/\zeta/u(c)$; 3) DRL-Router parameters α_0, ϵ , number of training episodes $T_{\max}$, and number of intervals/basis N ; and 4) database of travel time samples c_{ij} ;
Output: optimal routing policy π^* and full travel time distribution $Z^*(s, a)$ corresponding to π^* ;
Init: 1) policy π , 2) $Z^\pi(s, a)$, 3) counter $\leftarrow 0$, and 4) $\pi' \leftarrow \pi$;

```

1 while counter  $\leq T_{\max}$  do
2   Update  $\pi$  as adaptive double  $\epsilon$ -greedy with respect to  $\pi'$ ;
3    $t \leftarrow 0$ ;
4    $s_t \leftarrow \text{RandomChoice}(\mathcal{V})$ ;
5   while  $s_t \neq d$  do
6     get an action  $a_t$  using  $\pi$ ;
7     get the edge index (ij) corresponding to  $(s_t, a_t)$ ;
8     query the database to get  $K - 1$  travel time samples
9     update  $Z^\pi(s_t, a_t)$  through (8) with the  $K - 1$  database samples
       and  $t_{t+1}$ ;
10    update policy  $\pi'$  according to either (13) or (14);
11     $s_{t+1} = \mathcal{P}(s_t, a_t)$ ;
12     $t \leftarrow t + 1$ ;
13  counter  $\leftarrow$  counter + 1;
14   $\pi^* \leftarrow \pi'$ ;
15   $Z^*(s, a) \leftarrow Z^\pi(s, a)$ ;
16 Final.
```

There is a recent emerging trend of applying learning-based methods to solve a specific RSP problem.

want DRL-Router's runtime decision-making process to be "lightweight" in terms of the computational load and, hence, react in real time. We postpone the policy update and improvement process to the end of the episode, where the whole process can be done offline, with enough computation time.

DRL-Router Complexity Analysis

This section analyzes the computational complexity, memory complexity, and runtime decision-making complexity of DRL-Router. The computational cost of an algebraic operation is expressed as the number of floating-point operations (flops). A flop is defined as a subtraction, addition, or multiplication of two floating-point numbers [67]. To evaluate an algorithm's computational complexity, one needs to count the total number of flops, express it as a polynomial function of the dimensions of the involved matrices and vectors, and simplify the expression by ignoring all terms except the leading terms.

Computational Complexity Analysis

Examining Algorithm 1, we can see that the key computational load happens within the "while" loop (the initialization phase can be done offline.). With the inner "while" loop, lines 6 to 8 essentially perform a query over a set of $|\mathcal{E}| = m$ edges, whose computational complexity is $O(m)$; line 9 updates $Z^\pi(s, a)$ through (8), whose computational complexity is $O(KN)$. Supposing that the inner "while" loop executes H times, from line 6 to line 12, the total computational complexity is $O(mH + KNH)$. The outer "while" loop involves updating π to π' for all the edges based on $Z^\pi(s, a)$ and then deciding the best action. Examining (9) to (12), the calculation process of each obj value has a computational complexity of $O(N)$, and there are altogether m obj values to calculate; hence, the total computational complexity is $O(Nm)$. Thus, the total computational complexity for one execution of the outer "while" loop is $O(Nm)$.

Summing up the computational complexity for both the inner and outer "while" loops, and noting that the outer "while" loop operates $T_{\max}$ times, we have DRL-Router's computational complexity, at $O((mH + KNH + 2mN)T_{\max})$. From the computational complexity analysis process, we can see that DRL-Router scales roughly linearly with 1) the size of the graph (m), 2) the number of intervals/basis (N) in the Z^π representation, 3) the required number of travel time samples per edge (K), and 4) the total number of training episodes ($T_{\max}$).

Memory Complexity Analysis

Analyzing Algorithm 1, we can see that it requires computer memories to store the values, such as Z^π , c_{ij} , obj, and $\pi(s)$. Here, we analyze the main components' storage requirement. For $Z^\pi(s, a)$, in essence, we need to store only Z values for all

the edges; therefore, the total number of Z values is m , and for each Z value, we need to store N intervals/basis; thus, the required memory space is $O(mN)$. For the travel time database, we have m edges, and each edge is assumed with K travel time samples; thus, the memory space is $O(mK)$. For the obj value, it is a scalar for each edge; thus, the memory space is $O(m)$. For policy π , it stores the decisions for each vertex; thus, the required memory space is $O(n)$. Summing up, we have the space complexity of DRL-Router, at $O(m(N + K + 1) + n)$.

Runtime Computational Complexity Analysis

When deploying DRL-Router for real navigation services, it is important to analyze its response time to the user's request. During deployment, it is not necessary for DRL-Router to update and improve the routing policy. Instead, the ego vehicle needs only to make real-time decisions based on the currently estimated full travel time distribution, i.e., $Z^*(s, a)$. The key part is to calculate the corresponding obj(s, a) values for the current state and select the action/edge with the maximal (for the SOTA problem) and minimal (for other RSP problems) obj values. Calculating the obj values through one of the equations from (9) to (12) has the computational complexity of $O(N)$. Meanwhile, for most real-world applications, there are, on average, four or five edges connected with the current vertex. Thus, the runtime computational complexity of DRL-Router is $O(4N)$, which means that DRL-Router's runtime reaction time depends solely on the number of intervals/basis for Z distribution approximation.

Simulation Results and Comparisons

In this section, we evaluate DRL-Router's performance for the four aforementioned RSP planning objectives in a range of transportation networks and make comparisons with state-of-the-art RSP planning algorithms. Since state-of-the-art RSP planning algorithms usually target specific RSP problems, we lay down the different sets of benchmark algorithms for different RSP objectives in Table 3. Note that in Table 3, PQL [20] is the application of the canonical reinforcement learning algorithm to the SOTA problem, and CTD [22] is a newly proposed reinforcement learning algorithm for the mean-std shortest path problem. Therefore, PQL and CTD serve as the two most recently proposed reinforcement learning algorithms for the RSP problem. Meanwhile, we also apply the canonical reinforcement learning algorithm, namely

QL [68], to the RSP problem as one of the reinforcement learning baseline algorithms for comparison purposes.

In the following sections, we first construct a very simple test network to evaluate DRL-Router's correctness, applicability to different travel time distributions, and parameter sensitivity and then compare DRL-Router's performance and efficiency with state-of-the-art algorithms in four realistic transportation networks, namely, the Sioux Falls Network (SFN), Anaheim Network (AN), Winnipeg Network (WN), and Barcelona Network (BN). Finally, we showcase DRL-Router's adaptability to user-changed RSP objectives. Table 4 summarizes the related algorithms' parameter configurations. Note that the value inside the brackets ($\delta = 1$) refers to DOT's parameter setup for the AN, WN, and BN.

All the algorithms are implemented in Python 3.7, and we conduct the experiments on a 3.6-GHz, AMD Ryzen 5 3600, six-core desktop processor with the 64-b version of Windows 10 and 16 GB of random-access memory. The source code of DRL-Router as well as the related state-of-the-art algorithms is publicly available at <https://github.com/YoZo-X/DRL-Router>.

Design Parameter and Algorithm Correctness Study

In this section, we construct a very simple test network to 1) investigate the influence of different parameter con-

figurations, i.e., N and α_0 , on DRL-Router's performance; 2) evaluate DRL-Router's "correctness," i.e., whether DRL-Router converges to the true travel time distribution of a given routing policy; and 3) test DRL-Router's applicability to different travel time distribution models. Note that we do not investigate the influence of the number of travel time samples (K) because K is subject to the travel time sample availability, and DRL-Router simply performs better with larger K values, almost without incurring additional computational cost. The test network, as shown in Figure 1(a), consists of five vertices ($o = 1$, $d = 5$), and four edges, each of which follows a different travel time distribution, as given in Figure 1(b).

Learning Rate (α_0)

We first investigate the influence of learning rate α_0 with a fixed N , and Figure 2 details the convergence process of different learning rates. From the figure, we can see that with larger α_0 values, DRL-Router converges faster on

Table 3. The benchmark algorithms.

RSP Objective	Benchmark Algorithm
SOTA	PLM [11]
	DOT [7]
	OS-MIP [9]
	PQL [20]
	FMA [17]
	QL [68]
α -RSP	LR [30]
	MOM [28]
	E-PA [32]
	ILa [35]
Minimal mean-std	SP [26]
	GP3 [24]
	CTD [22]
	QL [68]
	Relax-1 [5]
MED	Algorithm CD-Path [47]

PLM: partial Lagrangian method; OS-MIP: one-stage mixed-integer programming; FMA: fourth moment approach; LR: Lagrangian relaxation method; MOM: moment matching method; E-PA: extended pulse algorithm; ILA: iterative labeling algorithm; SP: subgradient projection; GP3: Gaussian process path planning; Relax-1: relaxed pruning.

Table 4. The algorithm parameters.

Parameter	Description	Algorithm	Value
N	Number of basis	DRL-Router	51
α_0	Learning rate	DRL-Router, CTD, PQL	0.1
K	Travel time samples	DRL-Router	5
$T_{\max}$	Maximum number of episodes	DRL-Router, CTD, QL	2,000
MaxIt	Maximum number of iterations	OS-MIP, PLM, GP3 LR, SP, MOM	200
η	Duality gap	SP, LR, GP3	0.01
S	Number of travel time samples	OS-MIP, PLM, SP	50
δ	Time window duration	DOT	0.01 (1)
ϑ	Accuracy threshold	FMA	0.05

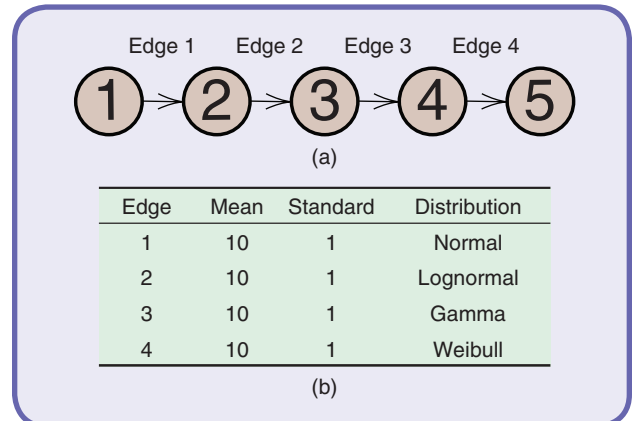


FIG 1 The (a) test network and (b) network information.

average but with higher variations (larger shaded areas) during learning. Considering the tradeoff between convergence speed and variance, we select $\alpha_0 = 0.1$ (as listed in Table 4) for the remaining experiments. Note that for $\alpha_0 = 0.1$ and $\alpha_0 = 0.5$, the curves coincide with each other after 150 training episodes; the underlying reason is that both α_0 values converge to the same performance metric after 150 episodes.

The Number of Basis (N)

Next, we investigate the influence of the number of basis (N). DRL-Router's performance with different N values is illustrated in Figure 3, from which we can see that the L_2 distance (a commonly used metric to gauge the dissimilarity between two distributions [69]) between the converged Z^π values and the true travel time distribution decreases as N increases, in general (we observe that when $N < 10$, the converged L_2 values are too large and even random;

thus, there is no such apparent decreasing trend.) However, the needed computation time increases. Balancing accuracy (a short L_2 distance) with efficiency (low computation time), we set $N = 51$ for the remaining experiments.

Correctness Evaluation

We evaluate DRL-Router's correctness and applicability to different travel time distribution models. Figure 4 presents a representative Z^π evolution process, with Figure 4(e) showing the "true" travel time distribution (estimated from the Monte Carlo estimation method). From the figure, we can see that DRL-Router evaluates the given policy's travel time distribution "correctly" [in terms of the distribution similarity between Figure 4(d) and (e)], and it applies to different kinds of travel time distribution models (we test four different distribution models, namely, normal, log normal, gamma, and the Weibull distribution, in the test network).

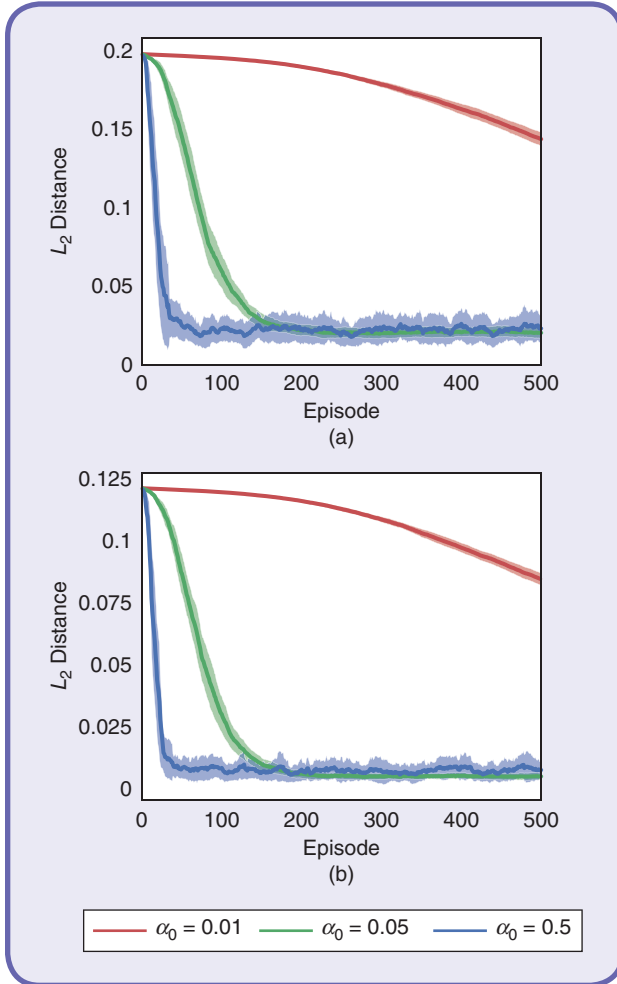


FIG 2 DRL-Router's convergence process for different learning rates. The y-axis refers to the L_2 distance between DRL-Router's estimated distribution when the ego vehicle starts at node 1 in Figure 1(a) and the true distribution (estimated from the Monte Carlo estimation method). Shaded areas indicate the standard deviations of the estimation. (a) $N = 31$. (b) $N = 51$.

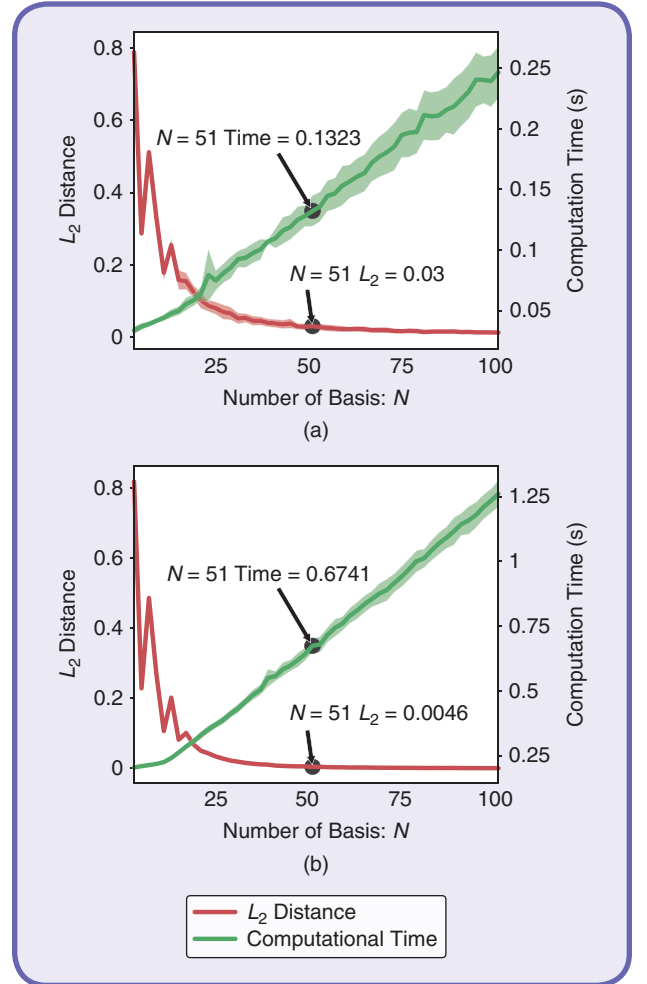


FIG 3 DRL-Router's performance (accuracy and efficiency) for different N values; (a) and (b) are results when we terminate the program at 100 episodes and 500 episodes. Shaded areas show the standard deviation of the performance indicators, i.e., the L_2 distance and computation time. (a) Epi = 100. (b) Epi = 500.

Performance and Efficiency Comparison in SFN, AN, WN, and BN

This section evaluates and compares DRL-Router's performance and efficiency with the state of the art in a range of realistic transportation networks, namely, the SFN, AN, WN, and BN. The test transportation network details, e.g., the number of vertices and edges, are presented in Table 5.

The edges' mean travel times of all four transportation networks are publicly available [70], and we adopt the same method as used in [25] to generate the standard deviation value of each edge in the corresponding network. Specifically, we set $\sigma_{ij} = \text{Uniform}(0, \kappa) \mu_{ij}$, where $\kappa = 0.4$. In the simulation setup, we assume that the edges' travel time follows a normal distribution. For the performance evaluation, we randomly generate 50 OD pairs with rejection (reject the OD pairs that are too near each other) for the AN, WN, and BN and five OD pairs for the SFN since it is a relatively small network, with the fewest number of nodes and edges. Then, we evaluate and compare the performance of DRL-Router with respective state-of-the-art algorithms for the four aforementioned RSP objectives, namely, SOTA, α -RSP, the mean-std shortest path, and MED. Specifically, for the SOTA objective, we set three types of path planning budget T for evaluation, namely, 1) the tight budget: $T = 0.975 \times t_{\text{LET}}$, where t_{LET} refers to the LET path's mean travel time; 2) exact budget: $T = t_{\text{LET}}$; and 3) loose budget:

$T = 1.025 \times t_{\text{LET}}$. For the α -RSP objective, we select three representative α values, namely, 1) risk averse: $\alpha = 0.9$, 2) risk neutral: $\alpha = 0.5$, and 3) risk seeking: $\alpha = 0.1$. For the mean-std objective, we select three representative ζ values as follows (the selection falls in the range of ζ values used in [35]): 1) $\zeta = 1$, 2) $\zeta = 10$, and 3) $\zeta = 40$. For the MED objective, we use the exponential disutility function, with $\alpha \in \{0.01, 0.05, 0.1\}$, which conforms to the parameter configurations used in [47]. To eliminate performance randomness, for each parameter configuration, we rerun DRL-Router 50 times and report the average performance.

Note that to compare the solution quality of each algorithm to each RSP problem in a unified view, we bring about a new concept of the optimality index (Opt), defined as

$$\text{Opt} = \frac{f_{\text{alg}} - f_{\text{LET}}}{f_{\text{LET}}} \times 100\% \quad (15)$$

Table 5. The test network details.

Test Network	Number of Edges (m)	Number of Vertices (n)
SFN	76	24
AN	914	416
WN	2,836	1,052
BN	2,522	1,020

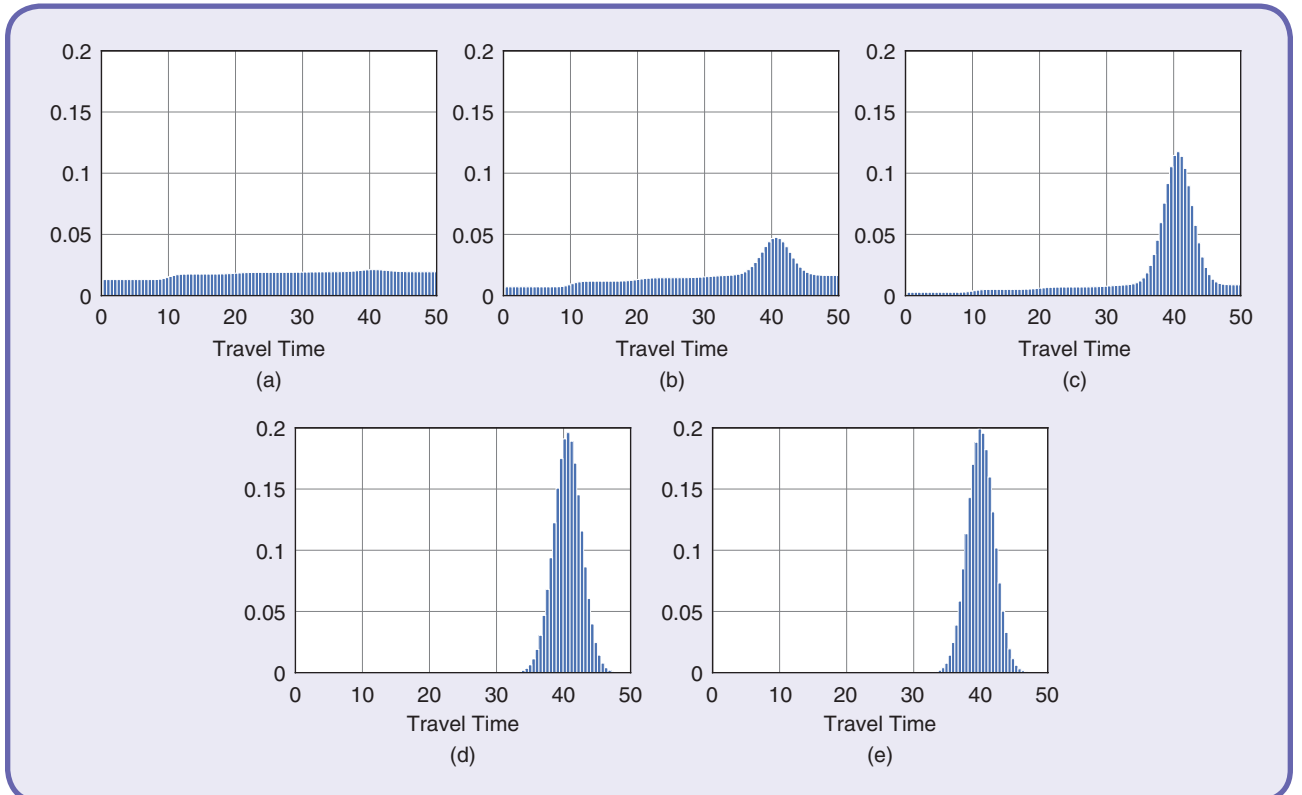


FIG 4 A representative evolution process of Z^π (the “true distribution” is estimated from the Monte Carlo method). (a) Epi = 20. (b) Epi = 50. (c) Epi = 100. (d) Epi = 500. (e) The true distribution.

for the SOTA problem and as

$$\text{Opt} = -\frac{f_{\text{alg}} - f_{\text{LET}}}{f_{\text{LET}}} \times 100\% \quad (16)$$

for the other three RSP problems. In (15) and (16), f_{alg} refers to the evaluating algorithm's average resulting RSP objective value; e.g., for SOTA, f_{alg} refers to the ego algorithm's average SOTA probability, and for mean-std, f_{alg} refers to the ego algorithm's average mean-std combination value. In addition, f_{LET} refers to the LET path's corresponding average RSP objective value; e.g., for SOTA, f_{LET} refers to the LET path's average SOTA probability. The reason we flip the sign between (15) and (16) is that for SOTA, the objective is to maximize the objective value, while for the other three RSP problems, the objective is to minimize the objective value. Through flipping the sign, the algorithm's Opt value is always better when it is larger, and it refers to the performance increase percentage with

respect to the LET path, which serves as the baseline performance indicator.

Figure 5 displays the comparative results of DRL-Router with the related baseline algorithms for the SOTA problem and the mean-std shortest path problem in the SFN, AN, WN, and BN. Note that we skip the performance and efficiency comparisons for the α -RSP problem and MED problem, as the figures are quite similar to the ones presented in Figure 5, and report the detailed performance metric comparisons per network and per parameter configuration in "Quantitative Comparison With Other RSP Objectives".

In Figure 5(a) and (c), we can see that DRL-Router achieves the top two performance metrics when compared with the respective set of benchmark algorithms in the four test networks. Note that for each evaluating network, we have already averaged three cases of T values for the SOTA problem and ζ values for the mean-std problem, respectively. For the computational efficiency comparison, it

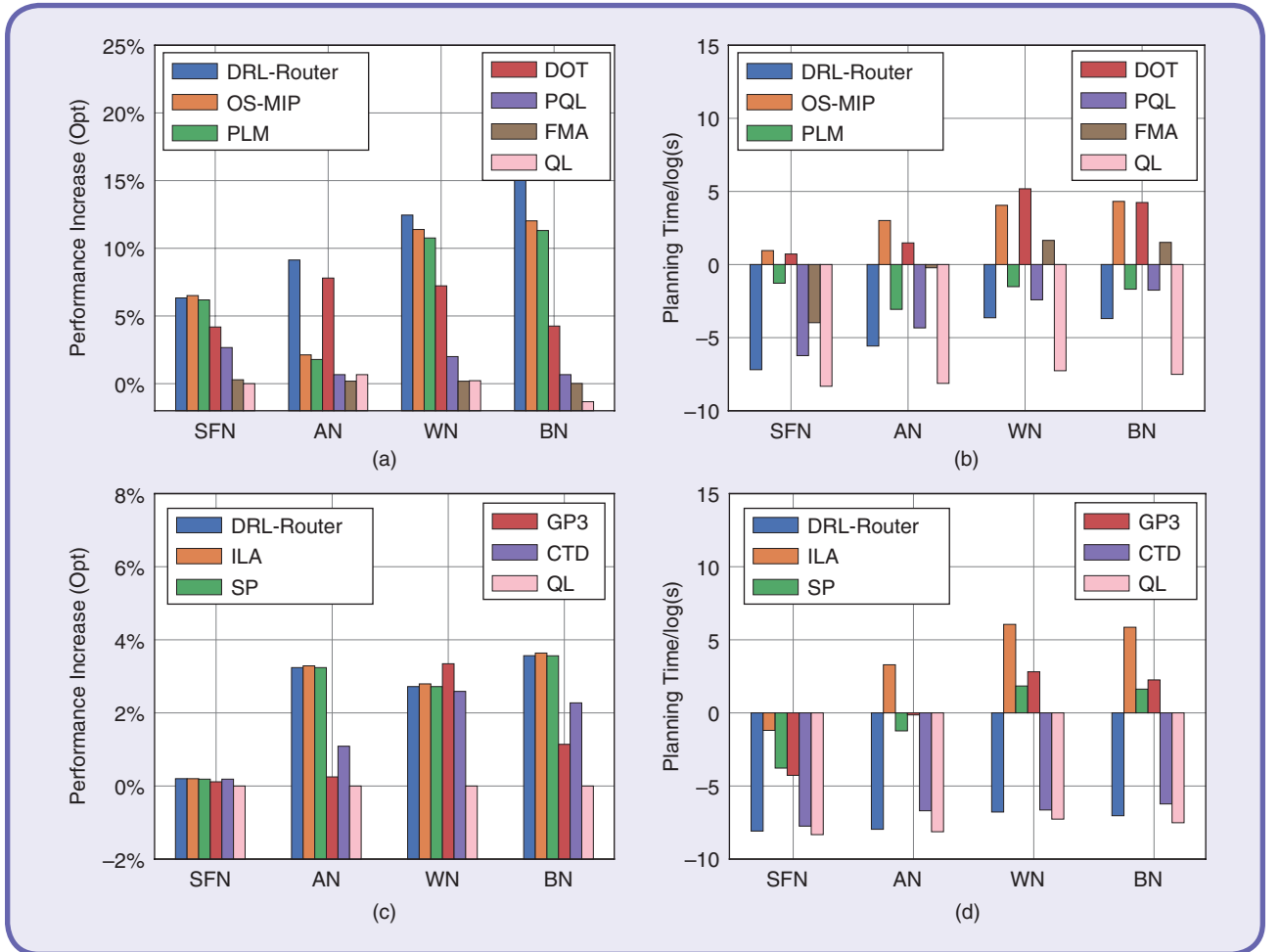


FIG 5 The performance and efficiency comparison of DRL-Router with baseline algorithms in the SFN, AN, WN, and BN for the SOTA problem and mean-std shortest path problem. [The units of the y-axis of (b) and (d) are logarithm values in seconds.] (a) The SOTA performance. (b) The SOTA efficiency. (c) The mean-std performance. (d) The mean-std efficiency.

Quantitative Comparison With Other RSP Objectives

This sidebar presents detailed quantitative comparisons of the distributional reinforcement learning (DRL)-based algorithm DRL-Router with the baseline algorithms for the remaining two reliable shortest path problems, i.e., the high-confidence constraint (α) and minimal expected

disutility (MED) path, for four realistic networks. For Tables S1 and S2, we report the algorithm's Opt value, which is the relative performance increase/decrease percentage with respect to the least expected time path's performance.

Table S1. The α -reliable result of the four networks.

Network	α	Performance Increase				Planning Time/Log (s)			
		DRL	LR	MOM	E-PA	DRL	LR	MOM	E-PA
SFN	0.9	2.17%	-17.87%	-2.63%	-2.63%	-7.02	-4.11	0.46	-5.91
	0.5	0%	-7.7%	-7.7%	-5.1%	-7.12	-3.98	0.48	-5.68
	0.1	1.06%	-2.18%	1.54%	-9.23%	-7.2	1.05	0.47	-4.84
AN	0.9	8.25%	8.12%	8.26%	0.04%	-6.78	1.67	2.83	-0.56
	0.5	8.29%	8.26%	8.32%	0.04%	-6.88	1.67	5.24	-0.56
	0.1	9.66%	8.31%	8.35%	0.02%	-6.99	1.67	5.24	-0.57
WN	0.9	19.44%	17.66%	18.69%	4.16%	-6.5	2.91	2.1	1.36
	0.5	21.99%	20.04%	20.05%	3.93%	-6.49	2.92	2.11	1.36
	0.1	24.9%	21.96%	21.62%	5.16%	-6.88	4.46	2.1	1.44
BN	0.9	51.57%	51.53%	51.59%	7.71%	-6.69	2.7	2.05	1.2
	0.5	51.07%	51.06%	51.07%	4.84%	-6.62	6.58	2.06	1.23
	0.1	50.4%	50.01%	50.38%	16.64%	-5.38	6.58	2.05	1.2

LR: Lagrangian relaxation method; MOM: moment matching method; E-PA: extended pulse algorithm; SFN: Sioux Falls Network; AN: Anaheim Network; WN: Winnipeg Network; BN: Barcelona Network.

Table S2. The MED result of the four networks.

Network	α	Performance Increase			Planning Time: Log (s)		
		DRL	Relax-1	Alg-CD	DRL	Relax-1	Alg-CD
SFN	0.01	0.12%	0%	-0.4%	-8.5	1.04	-4.59
	0.05	1.65%	-0.35%	-0.98%	-8.43	1.03	-4.53
	0.1	7.5%	-1.05%	-4.31%	-8.5	1.04	-4.59
AN	0.01	0.02%	0.36%	-0.5%	-8.43	1.94	-0.58
	0.05	0.14%	0.11%	-2.82%	-8.46	1.94	-0.58
	0.1	0.11%	0.02%	-3.17%	-8.41	1.94	-0.59
WN	0.01	0.03%	0.02%	-0.23%	-7.07	3.72	4.76
	0.05	0.05%	0.05%	-3.55%	-7.04	3.72	4.81
	0.1	0.02%	-0.25%	-12.85%	-7.08	3.73	5.09
BN	0.01	0.01%	0.01%	-0.3%	-7.33	3.5	2.69
	0.05	0.02%	-0.05%	-0.93%	-7.34	3.5	2.6
	0.1	0.13%	-0.11%	-2.78%	-7.33	3.5	2.93

Relax-1: relaxed pruning; Alg-CD: Algorithm CD-Path.

In practice, we need an algorithm that is able to update the distribution estimates based on bootstrapped estimates from the past.

can be seen from Figure 5(b) and (d) that for both the SOTA problem and the mean-std shortest path problem, and for all four transportation networks, DRL-Router is much faster than all the other benchmark algorithms by at least an order of magnitude (10 times faster). Note that in Figure 5(c), both ILP and SP exhibit quite good performances for the mean-std shortest path problem, and thus, the performance increases with respect to the LET path are similar between DRL-Router, ILP, and SP. For DRL-Router, the advantage is to have a “universal” solution applicable to all four RSP problems and to be fast in terms of the decision-making time. Therefore, similar performance with the state of the art for one specific RSP problem is quite acceptable.

In summary, DRL-Router achieves the top two performance metrics in all the evaluated RSP problems, which shows its universal application characteristic, and its decision-making time is at least an order of magnitude (10 times) faster than the related benchmark algorithms, which shows that it is fast.

DRL-Router Adaptability Evaluation

In the previous section, we showed DRL-Router’s universal and fast characteristics. This section evaluates its characteristic of being adaptive, i.e., how DRL-Router reacts in real time to a user-changed objective. Since, to the best of our knowledge, no state-of-the-art RSP solution offers adaptability to the switched-RSP objective, we do not lay down quantitative comparisons.

The term *adaptability* is quantified as the performance increase (or decrease) of DRL-Router with respect to the LET path’s performance when we train DRL-Router for one specific RSP objective, e.g., SOTA, but deploy it to another RSP objective, e.g., α -RSP. For example, we train DRL-Router for the SOTA problem with three T

values as specified in the previous section ($T = 0.975 \times t_{\text{LET}}$, $T = t_{\text{LET}}$, and $T = 1.025 \times t_{\text{LET}}$) but suppose that the user switches his or her objective to the mean-std shortest path objective, with $\zeta = 1, 10, 30$. How does the pre-trained DRL-Router for the SOTA problem behave in the mean-std setup? First, it is straightforward to apply the

pretrained SOTA DRL-Router, as it also offers a full distribution for a given routing policy. However, one cannot guarantee its optimality since it is trained with respect to the SOTA objective rather than the mean-std objective. We still deploy the pretrained DRL-Router to the mean-std setup and evaluate its performance by calculating its Opt value with respect to the LET path’s performance as stated in (15) or (16).

Table 6 lists the adaptation score matrix for the SFN, which is a 4×4 matrix, with element ij corresponding to the performance increase of DRL-Router when trained with the i th RSP objective and deployed with the j th RSP objective. For example, in the first row, second column, the element value of 0.38% means that when we train DRL-Router for the SOTA objective and deploy it for the α -RSP problem, the performance increase (Opt) with respect to the LET path is 0.38%. In Table 6, we can see that DRL-Router exhibits great adaptation ability, with all the elements being positive. Note that the values in the main diagonal of the adaptation score matrix refer to the average performance increase of DRL-Router when it is trained and deployed with respect to the same RSP objective.

Conclusion and Future Work

This article presented DRL-Router, a universal learning-based solution for the RSP problem. Different from state-of-the-art algorithms, DRL-Router 1) does not rely on pregauged travel time distribution models; 2) applies to all the prevailing RSP objectives, e.g., SOTA, α -reliable, mean-std, and MED; 3) is able to make decisions in real time; and 4) adapts smoothly with user-switched RSP objectives. We analyzed the computational complexity and runtime complexity of DRL-Router, and demonstrated the universal applicability of DRL-Router to different RSP objectives in a range of transportation networks.

Currently, DRL-Router offers an “all-to-one” service, which means that the well-trained DRL-Router is able to provide RSP routing services from “all” origin nodes to one fixed destination node. DRL-Router has to be trained completely separately for different destinations, which prohibits DRL-Router’s generalization ability to multiple destinations. In the future, we would like to incorporate the multitask learning paradigm into DRL-Router, which would endow its generalization ability. Meanwhile, we are keen

Table 6. The adaptation score matrix for the SFN.

	SOTA	Alpha	Mean-Std	MED
SOTA	6.33%	0.38%	0.09%	2.73%
Alpha	3.3%	1.06%	0.06%	3.31%
Mean-std	1.1%	0.82%	0.2%	3.08%
MED	3.3%	0.08%	0.08%	3.09%

on applying modern DRL techniques, e.g., the quantile regression deep Q-network [62], to increase the generalization ability of DRL-Router.

About the Authors



Hongliang Guo (guo_hongliang@i2r.a-star.edu.sg) earned his Ph.D. degree from the Stevens Institute of Technology, Hoboken, NJ, USA. He was a post-doctoral researcher in Almende, Nanyang Technological University and the Singapore-MIT Alliance for Research and Technology, respectively. Starting in 2023, he has been serving as an associate professor at College of Computer Science, Sichuan University (SCU), Chengdu, Sichuan 610065, China. His research interests include reliable path planning and learning under uncertainties as well as multirobot efficient search.



Wenda Sheng (ningenpre@gmail.com) earned his B.S. degree, with first-class honors, in 2020 from the University of Electronic Science and Technology of China, Chengdu 611731, China, where he is pursuing his M.S. degree in control science and engineering. His research interests include distributional reinforcement learning for autonomous navigation in dynamic environments and multirobot reliable search with factorized multiagent reinforcement learning.



Chen Gao (chengao982@gmail.com) earned his B.S. degree, with first-class honors, from the University of Electronic Science and Technology of China in 2020. He is pursuing his M.S. degree at ETH Zurich, 8092 Zurich, Switzerland. His research interests include integrating planning and learning, with applications to robotics and autonomous vehicles in unstructured environments.



Yaochu Jin (yaochu.jin@uni-bielefeld.de) earned his Ph.D. degree from Zhejiang University, China, and Dr.-Ing. degree from Ruhr University Bochum, Germany. He is an Alexander von Humboldt Professor for Artificial Intelligence at Bielefeld University, 33619 Bielefeld, Germany. His research interests include data-driven evolutionary optimization and multiobjective reinforcement learning. He is a Fellow of IEEE, the president-elect of the IEEE Computational Intelligence Society, and a member of Academia Europaea.

References

- [1] D. Brownstone, A. Ghosh, T. F. Golob, C. Kazimi, and D. Van Amelsfort, "Drivers' willingness-to-pay to reduce travel time: Evidence from the San Diego i-15 congestion pricing project," *Transp. Res. A, Policy Pract.*, vol. 37, no. 4, pp. 373–387, May 2003, doi: 10.1016/S0965-8564(02)00021-6.
- [2] H. Frank, "Shortest paths in probabilistic graphs," *Oper. Res.*, vol. 17, no. 4, pp. 583–599, Aug. 1969, doi: 10.1287/opre.17.4.583.
- [3] A. Chen and Z. Ji, "Path finding under uncertainty," *J. Adv. Transp.*, vol. 39, no. 1, pp. 19–37, Jan. 2010, doi: 10.1002/atr.5670390104.
- [4] T. Uchida and Y. Iida, "Risk assignment: A new traffic assignment model considering the risk of travel time variation," in *Proc. Transp. Traffic Theory*, 1993, pp. 89–105.
- [5] I. Murthy and S. Sarkar, "Stochastic shortest path problems with piecewise-linear concave utility functions," *Manage. Sci.*, vol. 44, no. 11-part-2, pp. S125–S136, Nov. 1998, doi: 10.1287/mnsc.44.11.S125.
- [6] F. Manseur, N. Farhi, C. N. Van Phu, H. Haj-Salem, and J.-P. Lebacque, "Robust routing, its price, and the tradeoff between routing robustness and travel time reliability in road networks," *Eur. J. Oper. Res.*, vol. 285, no. 1, pp. 159–171, Aug. 2020, doi: 10.1016/j.ejor.2018.10.053.
- [7] A. Arun Prakash, "Algorithms for most reliable routes on stochastic and time-dependent networks," *Transp. Res. B, Methodological*, vol. 138, pp. 202–220, Aug. 2020, doi: 10.1016/j.trb.2020.05.015.
- [8] A. Agafonov, V. Myasnikov, and A. Maksimov, "Approximation of the road segments travel time using Lévy distributions in the reliable shortest path problem," *J. Phys., Conf. Ser.*, vol. 1368, no. 3, Nov. 2019, Art. no. 032008, doi: 10.1088/1742-6596/1368/3/032008.
- [9] L. Yang and X. Zhou, "Optimizing on-time arrival probability and percentile travel time for elementary path finding in time-dependent transportation networks: Linear mixed integer programming reformulations," *Transp. Res. B, Methodological*, vol. 96, pp. 68–91, Feb. 2017, doi: 10.1016/j.trb.2016.11.012.
- [10] L. Shen, H. Shao, L. Zhang, and J. Zhao, "The global optimal algorithm of reliable path finding problem based on backtracking method," *Math. Problems Eng.*, vol. 2017, pp. 1–10, Oct. 2017, doi: 10.1155/2017/4586471.
- [11] Z. Cao, H. Guo, J. Zhang, D. Niyato, and U. Fastenrath, "Improving the efficiency of stochastic vehicle routing: A partial Lagrange multiplier method," *IEEE Trans. Veh. Technol.*, vol. 65, no. 6, pp. 3993–4005, Jun. 2016, doi: 10.1109/TVT.2015.2480964.
- [12] Z. Cao, H. Guo, J. Zhang, D. Niyato, and U. Fastenrath, "Finding the shortest path in stochastic vehicle routing: A cardinality minimization approach," *IEEE Trans. Intell. Transp. Syst.*, vol. 17, no. 6, pp. 1688–1702, Jun. 2016, doi: 10.1109/TITS.2015.2498160.
- [13] G. Sabran, S. Samaranayake, and A. Bayen, "Precomputation techniques for the stochastic on-time arrival problem," in *Proc. Meeting Algorithm Eng. Exp. (ALENEX)*, Society for Industrial and Applied Mathematics, 2014, pp. 138–146.
- [14] Y. M. Nie and X. Wu, "Shortest path problem considering on-time arrival probability," *Transp. Res. B, Methodological*, vol. 43, no. 6, pp. 597–613, Jul. 2009, doi: 10.1016/j.trb.2009.01.008.
- [15] Y. Fan, R. Kalaba, and J. Moore, "Arriving on time," *J. Optim. Theory Appl.*, vol. 127, no. 3, pp. 497–513, Dec. 2005, doi: 10.1007/s10957-005-7498-5.
- [16] C. Gao, H. Guo, and W. Sheng, "GP4: Gaussian process proactive path planning for the stochastic on time arrival problem," *IEEE Trans. Veh. Technol.*, vol. 70, no. 10, pp. 9849–9862, Oct. 2021, doi: 10.1109/TVT.2021.3109169.
- [17] H. Guo, Z. He, C. Gao, and D. Rus, "Navigation with time limits in transportation networks: A fourth moment approach," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 12, pp. 23,781–23,796, Dec. 2022, doi: 10.1109/TITS.2022.3189865.
- [18] Z. Cao et al., "Improving the performance of transportation networks: A semi-centralized pricing approach," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 10, pp. 6353–6364, Oct. 2021, doi: 10.1109/TITS.2020.2991759.
- [19] Z. Wang, K. You, S. Song, and Y. Zhang, "Wasserstein distributionally robust shortest path problem," *Eur. J. Oper. Res.*, vol. 284, no. 1, pp. 31–43, Jul. 2020, doi: 10.1016/j.ejor.2020.01.009.
- [20] Z. Cao et al., "Using reinforcement learning to minimize the probability of delay occurrence in transportation," *IEEE Trans. Veh. Technol.*, vol. 69, no. 3, pp. 2424–2436, Mar. 2020, doi: 10.1109/TVT.2020.2964784.
- [21] T. Xing and X. Zhou, "Finding the most reliable path with and without link travel time correlation: A Lagrangian substitution based approach," *Transp. Res. B, Methodological*, vol. 45, no. 10, pp. 1660–1679, Dec. 2011, doi: 10.1016/j.trb.2011.06.004.
- [22] H. Guo, X. Hou, and Q. Peng, "CTD: Cascaded temporal difference learning for the mean-standard deviation shortest path problem," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 8, pp. 10,868–10,886, Aug. 2022, doi: 10.1109/TITS.2021.3096829.

- [23] Y. Pan, "A nonlinear mixed integer programming method to find the reliable shortest path in a stochastic network," in *Proc. 16th COTA Int. Conf. Transp. Prof.*, 2016, pp. 71–77, doi: 10.1061/9780784479896.007.
- [24] H. Guo, X. Hou, Z. Cao, and J. Zhang, "GP3: Gaussian process path planning for reliable shortest path in transportation networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 8, pp. 11,575–11,590, Aug. 2022, doi: 10.1109/TITS.2021.3105415.
- [25] Y. Zhang and A. Khani, "An algorithm for reliable shortest path problem with travel time correlations," *Transp. Res. B, Methodological*, vol. 121, no. C, pp. 92–113, Oct. 2019, doi: 10.1016/j.trb.2018.12.011.
- [26] Y. Zhang, Z.-J. M. Shen, and S. Song, "Lagrangian relaxation for the reliable shortest path problem with correlated link travel times," *Transp. Res. B, Methodological*, vol. 104, pp. 501–521, Oct. 2017, doi: 10.1016/j.trb.2017.04.006.
- [27] Q. Tu, L. Cheng, T. Yuan, Y. Cheng, and M. Li, "The constrained reliable shortest path problem for electric vehicles in the urban transportation network," *J. Cleaner Prod.*, vol. 261, Mar. 2020, Art. no. 121150, doi: 10.1016/j.jclepro.2020.121150.
- [28] P. Chen, R. Tong, G. Lu, and Y. Wang, "The α -reliable path problem in stochastic road networks with link correlations: A moment-matching-based path finding algorithm," *Expert Syst. Appl.*, vol. 110, pp. 20–32, Nov. 2018, doi: 10.1016/j.eswa.2018.05.022.
- [29] P. Chen, R. Tong, B. Yu, and Y. Wang, "Reliable shortest path finding in stochastic time-dependent road network with spatial-temporal link correlations: A case study from Beijing," *Expert Syst. Appl.*, vol. 147, Jun. 2020, Art. no. 115192, doi: 10.1016/j.eswa.2020.115192.
- [30] W. Zeng, T. Miwa, Y. Wakita, and T. Morikawa, "Application of Lagrangian relaxation approach to α -reliable path finding in stochastic networks with correlated link travel times," *Transp. Res. C, Emerg. Technol.*, vol. 56, pp. 309–354, Jul. 2015, doi: 10.1016/j.trc.2015.04.018.
- [31] F. Guerriero and P. Beraldi, "The α -reliable shortest path problem," *Algorithmic Oper. Res.*, vol. 3, no. 1, pp. 59–66, Mar. 2008.
- [32] D. Corredor-Montenegro, N. Cabrera, R. Akhavan-Tabatabaei, and A. L. Medaglia, "On the shortest α -reliable path problem," *J. Spanish Soc. Statist. Oper. Res.*, vol. 29, no. 1, pp. 287–318, Mar. 2021, doi: 10.1007/s11750-021-00592-5.
- [33] S. Biswal, G. Ghorai, and S. Mohanty, " α -reliable shortest path problem in uncertain time-dependent networks," *Int. J. Appl. Comput. Math.*, vol. 8, no. 4, pp. 1–12, Aug. 2022, doi: 10.1007/s40819-022-01531-z.
- [34] M. Ruß, G. Gust, and D. Neumann, "The constrained reliable shortest path problem in stochastic time-dependent networks," *Oper. Res.*, vol. 69, no. 3, pp. 709–726, Apr. 2021, doi: 10.1287/opre.2020.2089.
- [35] A. Khani and S. D. Boyles, "An exact algorithm for the mean-standard deviation shortest path problem," *Transp. Res. B, Methodological*, vol. 81, pp. 252–266, Nov. 2015, doi: 10.1016/j.trb.2015.04.002.
- [36] Y. Liu, S. Blandin, and S. Samaranayake, "Stochastic on-time arrival problem in transit networks," *Transp. Res. B, Methodological*, vol. 119, pp. 122–158, Jan. 2019, doi: 10.1016/j.trb.2018.11.013.
- [37] B. Y. Chen, C. Shi, J. Zhang, W. H. Lam, Q. Li, and S. Xiang, "Most reliable path-finding algorithm for maximizing on-time arrival probability," *Transportmetrica B, Transport Dyn.*, vol. 5, no. 3, pp. 248–264, Jun. 2017, doi: 10.1080/21680566.2016.1169953.
- [38] B. Y. Chen, W. H. Lam, and Q. Li, "Efficient solution algorithm for finding spatially dependent reliable shortest path in road networks," *J. Adv. Transp.*, vol. 50, no. 7, pp. 1413–1431, Sep. 2016, doi: 10.1002/atr.1408.
- [39] H. Guo, R. Shi, D. Rus, and W.-Y. Yau, "Dual dynamic programming for the mean standard deviation Canadian traveller problem," *IEEE Trans. Veh. Technol.*, vol. 71, no. 11, pp. 11,465–11,479, Nov. 2022, doi: 10.1109/TVT.2022.3191490.
- [40] Y. Fan and Y. Nie, "Optimal routing for maximizing the travel time reliability," *Netw. Spatial Econ.*, vol. 6, nos. 3–4, pp. 333–344, Sep. 2006, doi: 10.1007/s11067-006-9287-6.
- [41] X. Hou, H. Guo, and Y. Zhang, "Adaptive reliable shortest path in Gaussian process regulated environments," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2020, pp. 6819–6825, doi: 10.1109/IROS45743.2020.9341736.
- [42] M. Shahabi, A. Unnikrishnan, and S. D. Boyles, "An outer approximation algorithm for the robust shortest path problem," *Transp. Res. E, Logistics Transp. Rev.*, vol. 58, pp. 52–66, Nov. 2013, doi: 10.1016/j.tre.2013.07.002.
- [43] S. Sen, R. Pillai, S. Joshi, and A. K. Rathi, "A mean-variance model for route guidance in advanced traveler information systems," *Transp. Sci.*, vol. 35, no. 1, pp. 37–49, Feb. 2001, doi: 10.1287/trsc.35.1.37.10141.
- [44] R. A. Sivakumar and R. Batta, "The variance-constrained shortest path problem," *Transp. Sci.*, vol. 28, no. 4, pp. 309–316, Nov. 1994, doi: 10.1287/trsc.28.4.309.
- [45] M. Rajabi-Bahaabadi, A. Shariat-Mohaymany, M. Babaei, and C. W. Ahn, "Multi-objective path finding in stochastic time-dependent road networks using non-dominated sorting genetic algorithm," *Expert Syst. Appl.*, vol. 42, no. 12, pp. 5056–5064, Jul. 2015, doi: 10.1016/j.eswa.2015.02.046.
- [46] Z. Ji, Y. S. Kim, and A. Chen, "Multi-objective α -reliable path finding in stochastic networks with correlated link costs: A simulation-based multi-objective genetic algorithm approach (SMOGA)," *Expert Syst. Appl.*, vol. 38, no. 3, pp. 1515–1528, Mar. 2011, doi: 10.1016/j.eswa.2010.07.064.
- [47] H. Huang and S. Gao, "Optimal paths in dynamic networks with dependent random link travel times," *Transp. Res. B, Methodological*, vol. 46, no. 5, pp. 579–598, Jun. 2012, doi: 10.1016/j.trb.2012.01.005.
- [48] S. Gao and I. Chabini, "Optimal routing policy problems in stochastic time-dependent networks," *Transp. Res. B, Methodological*, vol. 40, no. 2, pp. 95–122, Feb. 2006, doi: 10.1016/j.trb.2005.02.001.
- [49] H. Hu and R. Sotirov, "On solving the quadratic shortest path problem," *INFORMS J. Comput.*, vol. 32, no. 2, pp. 219–233, Spring 2020, doi: 10.1287/ijoc.2018.0861.
- [50] B. Rostami et al., "The quadratic shortest path problem: Complexity, approximability, and solution methods," *Eur. J. Oper. Res.*, vol. 268, no. 2, pp. 473–485, Feb. 2018, doi: 10.1016/j.ejor.2018.01.054.
- [51] A. Caplin and J. Leahy, "Psychological expected utility theory and anticipatory feelings," *Quart. J. Econ.*, vol. 116, no. 1, pp. 55–79, Feb. 2001, doi: 10.1162/003355501556347.
- [52] S. Lim, H. Balakrishnan, D. Gifford, S. Madden, and D. Rus, "Stochastic motion planning and applications to traffic," *Int. J. Robot. Res.*, vol. 30, no. 6, pp. 699–712, May 2011, doi: 10.1177/0278364910386259.
- [53] K. K. Srinivasan, A. Prakash, and R. Seshadri, "Finding most reliable paths on networks with correlated and shifted log-normal travel times," *Transp. Res. B, Methodological*, vol. 66, pp. 110–128, Aug. 2014, doi: 10.1016/j.trb.2013.10.011.
- [54] S. Susilawati, M. A. Taylor, and S. V. Somenahalli, "Distributions of travel time variability on urban roads," *J. Adv. Transp.*, vol. 47, no. 8, pp. 720–736, 2015, doi: 10.1002/atr.192.
- [55] E. Wong and A. Khani, "Transit delay estimation using stop-level automated passenger count data," *J. Transp. Eng. A, Syst.*, vol. 144, no. 3, pp. 2473–2907, Mar. 2018.
- [56] M. Aron, B. Neila, and Y. Guessous, "Estimating travel time distribution for reliability analysis," in *Proc. Transport Res. Arena*, Jan. 2014, pp. 1–10.
- [57] G. Li, L. Boukhatem, and J. Wu, "Adaptive quality-of-service-based routing for vehicular ad hoc networks with ant colony optimization," *IEEE Trans. Veh. Technol.*, vol. 66, no. 4, pp. 3249–3264, Apr. 2017, doi: 10.1109/TVT.2016.2586382.
- [58] W. Zhou, J. Fu, H. Yan, X. Du, Y. Wang, and H. Zhou, "Event-triggered approximate optimal path-following control for unmanned surface vehicles with state constraints," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 1, pp. 104–118, Jan. 2023, doi: 10.1109/TNNLS.2021.3090054.
- [59] H. Song, J. Bai, Y. Yi, J. Wu, and L. Liu, "Artificial intelligence enabled internet of things: Network architecture and spectrum access," *IEEE Comput. Intell. Mag.*, vol. 15, no. 1, pp. 44–51, Feb. 2020, doi: 10.1109/MCI.2019.2954645.
- [60] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 2018.
- [61] M. G. Bellemare, W. Dabney, and R. Munos, "A distributional perspective on reinforcement learning," in *Proc. Int. Conf. Mach. Learn. (ICML)*, PMLR, 2017, pp. 449–458.
- [62] W. Dabney, M. Rowland, M. Bellemare, and R. Munos, "Distributional reinforcement learning with quantile regression," in *Proc. 32nd AAAI Conf. Artif. Intell. (AAAI)*, 2018, pp. 2892–2901, doi: 10.1609/aaai.v32i1.11791.
- [63] W. Dabney, G. Ostrovski, D. Silver, and R. Munos, "Implicit quantile networks for distributional reinforcement learning," in *Proc. Int. Conf. Mach. Learn. (ICML)*, PMLR, 2018, pp. 1096–1105.
- [64] D. Yang, L. Zhao, Z. Lin, T. Qin, J. Bian, and T.-Y. Liu, "Fully parameterized quantile function for distributional reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, Curran Associates, Inc., 2019, vol. 32, pp. 6161–6170.
- [65] X. Ma, Q. Zhang, L. Xia, Z. Zhou, J. Yang, and Q. Zhao, "Distributional soft actor critic for risk sensitive learning," 2020, *arXiv:2004.14547*.
- [66] R. Peyre, "Comparison between W2 distance and H^{-1} norm, and localisation of Wasserstein distance," in *Proc. ESAIM, Contr. Optim. Calculus Variations (ESAIM: COCV)*, 2018, pp. 1489–1501.
- [67] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge, U.K.: Cambridge Univ. Press, 2004.
- [68] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Mach. Learn.*, vol. 8, nos. 3–4, pp. 279–292, May 1992, doi: 10.1007/BF00992698.
- [69] J. Chung, P. Kannappan, C. T. Ng, and P. Sahoo, "Measures of distance between probability distributions," *J. Math. Anal. Appl.*, vol. 138, no. 1, pp. 280–292, Feb. 1989, doi: 10.1016/0022-247X(89)90353-1.
- [70] H. Bar-Gera, "Transportation network test problems," GitHub. Accessed: Nov. 28, 2022. [Online]. Available: <https://github.com/bstabler/TransportationNetworks>